

LABORATORY WORK N1

HASKELL PROGRAMMING LANGUAGE

Note: Each task must be executed in Haskell and provided output with proves that selected code works. Laboratory works will be checked via **Turn It In** – copypaste_from_internet_solutions = 0.

Hints:

1. Comments in the code are welcome
2. The more complex&robust algorithm is the bigger grade you will receive
3. Don't hesitate to add any checks / debugging to existing solutions, it also will contribute to final grade

TASK 0.

Download and install Haskell compiler on your machine.

TASK 1.

Create function to detect the maxim number from list of numbers.

```
[5, 10, 11, 15, 3, 8];
```

Variant 1-5. Solution 1.

- sort the list descending
- print the first number

Requirement: Function should return meaningful output with short description, e.g. "The maxim number is XXX"

Variant 6-10. Solution 2.

Iterate over all the numbers in the list and save the largest one or use appropriate function

Requirement: Function should return meaningful output with short description, e.g. "The maxim number is XXX"

TASK 2.

Define a function that reverses a simple name

Variant 1-5. Solution 1.

Define function with the following functionality: having a name, compute and print the reverse version. Function should be able to accept string with input (name)

- split the string into an array of chars
- sort that in reverse
- join the letters back

Variant 6-10. Solution 2.

Loop back from the last character and compute the new text. Function should be able to accept string with input

TASK 3.

Define a function with the following functionality:

Capitalize the First Letter of Each Word in a Sentence.

Function should be able to accept string with input

Variant 1,6. Solution 1.

Given a text compute and print the same sentence with all words having the first letter capitalized.

- split the sentence in words by the space character
- capitalize each word
- join all the words back separated by a single space character

Variant 2,3,4,5,7,8,9,10. Solution 2.

Write a reducer that computes the capitalized text. It takes the current text and a new word and returns a new text with the capitalized word appended.

TASK 4.

Variant 1, 6.

Define a function that returns the one but last element of a list.

Variant 2-3.

Write a function *numbOfSol* that, for some h , i , and j , determines the number of solutions of the equation $hx^2 + ix + j = 0$, using case distinction.

What is the type of the following functions? *tail*, *sqrt*, *pi*, *exp*, *(^)*, *(/=)* and *numbOfSol*? How can you query the interpreter for the type of an expression and how can you explicitly specify the types of functions in your program?

Variant 4-5.

A recursive function is only sensible if the condition is that the value of its parameters becomes simpler in each recursive application is met. Consider the following definition of the *factorial* function:

```
fac n | n == 0 = 1
      | otherwise = n * fac (n - 1)
```

- What happens if you evaluate `fac (-3)`?
- How can you formulate the condition more precisely?

Variant 7-8.

Define a function which returns the last element of a list.

Variant 9-10.

Given the following definitions:

```
thrice a = [a, a, a]
```

```
sums (a : b : bs) = a : sums (a + b : bs)
sums as           = as
```

What does the following expression evaluate to?

```
map thrice (sums [0 .. 4])
```

TASK 5.

Variant 1.

Define an operator `(!!)` which returns the `i`-th element of a list

Variant 2.

Define a function that determines the “run-length encoding” of a list: `[1, 2, 2, 3, 2, 4]` is mapped to `[(1, 1), (2, 2), (1, 3), (1, 2), (1, 4)]`. That is, the list is mapped to a list of pairs whose first element says how many times the second component of the pair appears in adjacent positions in the list.

Variant 3.

Define the function `concat :: [[a]] -> [a]` which flattens a list of lists: `concat [[1, 2], [3], [], [4, 5]]` evaluates to `[1, 2, 3, 4, 5]`.

Variant 4.

Define a function `remSuccessiveDuplicates` which removes successive repeated elements from a list: `[1, 2, 2, 3, 2, 4]` is mapped to `[1, 2, 3, 2, 4]`.

Variant 5.

Define a function that groups successive duplicate elements in a list into sublists: `[1, 2, 2, 3, 2, 4]` is mapped to `[[1], [2, 2], [3], [2], [4]]`.

Variant 6.

Define a function that determines whether a list is a palindrome, that is, whether the list is equal to its reversal.

Variant 7-8.

Define a function which constructs the original list given its run-length-encoded version.

Variant 9-10.

Write a data type `Complex` to represent complex numbers of the form $a + b \cdot i$. Write its `Num` instance.

TASK 6.

Variant 1, 6.

Write a function `gaps` that gives all the possibilities to take out one element from a list. For example:

```
gaps [1,2,3,4,5] = [[2,3,4,5], [1,3,4,5], [1,2,4,5], [1,2,3,5], [1,2,3,4]]
```

Variant 2-3.

Write a function which takes two lists and removes all the elements from the second list from the first list.

Variant 4-5.

Implement the function `split` with the following type: `split :: Int -> [a] -> [[a]]`. This function divides the given list in sublists, where the sublists have the given length. Only the last list might be shorter.

Variant 7-8.

We can represent a matrix as a list of lists of the same length. Write a function `transpose :: [[a]] -> [[a]]` which maps the *i*-th element of the *j*-th list to the *j*-th element of the *i*-th list. Hint: you can make use of the function:

```
zipWith op (x:xs) (y:ys) = (x `op` y) : zipWith xs ys
zipWith op _      _      = [ ]
```

Variant 9-10.

Prove the following law about list functions.

```
length . map f = length
```

TASK 7.

Consider the following datatype definition for binary trees that we shall want to use to implement binary search trees:

```
data Tree a = Branch a (Tree a) (Tree a) | Leaf
```

Write a function `isSearchTree :: Tree a -> Bool` that verifies that its argument is a binary search tree.

Then test the property that given a binary search tree *t*, inserting a value into the tree results in yet another binary search tree. The code for inserting a new value into the tree is:

```
insertTree :: Ord a => a -> Tree a -> Tree a
insertTree e Leaf = Branch e Leaf Leaf
insertTree e (Branch x li re)
  | e <= x = Branch x (insertTree e li) re
  | e > x  = Branch x li (insertTree e re)
```

Experiment with mutating the implementation of `insertTree` to find out whether your property can in fact discover that the mutated implementation no longer maps binary search trees to binary search trees.

TASK 8.

Prove the following law about list functions.

```
map f . concat = concat . map (map f)
```